Regular vs TinyML Model Comparison

This document defines the comparison between a regular model trained from scratch, the YAMMnet transfer learning model and a distilled student model. The comparison should be small, reproducible, and directly connected to the safety-monitoring use case.

Research Question

How can a strong regular audio model help train a compact quantized TinyML model that can run on the ESP32 audio node?

Task

Classify short audio windows into selected real sound-event classes that fit the safety-monitoring context.

The notebook uses ESC-50 automatically, clones the Donate-a-cry corpus from GitHub, and can add UrbanSound8K and FSD50K when those audio folders are present. This keeps the first run reproducible while supporting the more useful final label split with gunshot, thud, screaming, alarm, and mechanical false-positive classes.

Target Labels (v6+, 12 classes)

```
glass_breaking
gunshot
explosion_or_fireworks
impact_or_thud
scream_or_shout
siren_or_alarm
footsteps
crying_or_sobbing
pet_noise
weather_noise
mechanical_noise
unknown_background
```

`household_noise` was dissolved into `unknown_background` in v6 because household sounds are acoustically inconsistent as a single class and the category had weak performance. `unknown_background` is an explicit escape valve — events the model does not recognise are labeled there rather than forced into a wrong safety-relevant class. Risk scores are assigned per-category in firmware/openHAB, not in the model, so keeping `impact_or_thud` clean (no household slams mixed in) is essential.

Useful verified source labels (v7):

Target label	Dataset labels to use
`glass_breaking`	ESC-50 `glass_breaking`; FSD50K `Glass`, `Shatter`, `Chink_and_clink`

Target label	Dataset labels to use
gunshot	UrbanSound8K gun_shot; FSD50K Gunshot_and_gunfire
explosion_or_fireworks	ESC-50 fireworks; FSD50K Explosion, Fireworks, Boom
impact_or_thud	FSD50K Thump_and_thud, Crack, Hammer
scream_or_shout	FSD50K Screaming, Shout, Yell, Screech
siren_or_alarm	ESC-50 siren, clock_alarm; UrbanSound8K siren; FSD50K Siren, Alarm, Doorbell, Ringtone
footsteps	ESC-50 footsteps; FSD50K Walk_and_footsteps, Run
crying_or_sobbing	ESC-50 crying_baby; FSD50K Crying_and_sobbing; Donate-a-cry corpus (~450 wav, auto-cloned)
pet_noise	ESC-50 dog, cat; UrbanSound8K dog_bark; FSD50K Dog, Bark, Cat, Meow, Purr, Growling, Domestic_animals_and_pets
weather_noise	ESC-50 rain, thunderstorm, water_drops, wind, sea_waves, crickets; FSD50K Rain, Raindrop, Thunder, Thunderstorm, Wind, Stream, Ocean, Waves_and_surf
mechanical_noise	ESC-50 chainsaw, engine, helicopter, train, airplane, hand_saw; UrbanSound8K drilling, jackhammer, engine_idling; FSD50K Drill, Power_tool, Sawing, Tools, Mechanical_fan, Engine, Motor_vehicle_(road), Aircraft, Motorcycle, Bus, Truck, Train, Rail_transport, Idling, Vehicle
unknown_background	ESC-50 vacuum_cleaner, washing_machine, clock_tick, laughing, clapping, keyboard_typing, snoring, chirping_birds, crow, crackling_fire; FSD50K Domestic_sounds_and_home_sounds, Cupboard_open_or_close, Drawer_open_or_close, Microwave_oven, Computer_keyboard, Typing, Conversation, Speech, Music, Laughter, Chatter, Crowd, Clapping, Applause, Singing, Female_singing, Male_singing, Slam, Knock, Telephone, Water, Hiss, Giggle, Cheering, Fart; UrbanSound8K air_conditioner, car_horn, children_playing, street_music

The model predicts the acoustic event class. The ESP32 firmware or openHAB rules later map that class to a risk category. This avoids training the model on categories that are semantically useful but acoustically inconsistent.

Example later mapping:

```
glass_breaking          -> critical_impulse  -> high risk
gunshot                 -> critical_impulse  -> critical risk
explosion_or_fireworks  -> explosive_context -> medium/high risk depending on
context
impact_or_thud          -> impact            -> medium/high risk depending on
```

```
confidence
scream_or_shout         -> distress          -> high risk
siren_or_alarm          -> alarm_context     -> medium/high risk depending on
confidence
weather_noise           -> weather_context   -> low risk modifier
pet_noise               -> ignore_or_low     -> no direct alarm
unknown_background      -> ignore            -> no direct alarm
```

The class-to-risk mapping is not part of model training. It belongs in firmware/openHAB logic so that risk policy can change without retraining the model.

Notebook

The current notebooks are versioned:

```
tinyml/notebooks/sound_classification_v8.ipynb   <- best completed run (student
66.9% int8)
tinyml/notebooks/sound_classification_v9.ipynb   <- Mixup regression (student
64.0%)
tinyml/notebooks/sound_classification_v10.ipynb  <- current (three-phase teacher,
BatchNorm, SpecAugment student)
```

It uses a laptop/Colab-side audio dataset so results are reproducible without depending on live microphone input. The INMP441 microphone is soldered and working; the TinyML model is not yet deployed on the ESP32.

Development setup:

```
VS Code + local .venv kernel for editing/light runs
Google Colab runtime for heavier TensorFlow training
```

The local kernel is documented in `tinyml/README.md` as `Safety TinyML (.venv)`. The Colab route uses the same notebook, so results and generated artifacts remain comparable.

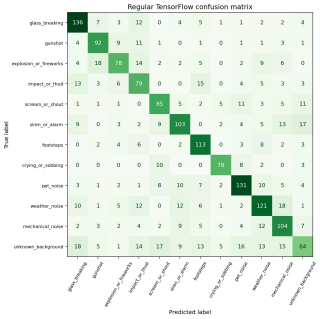
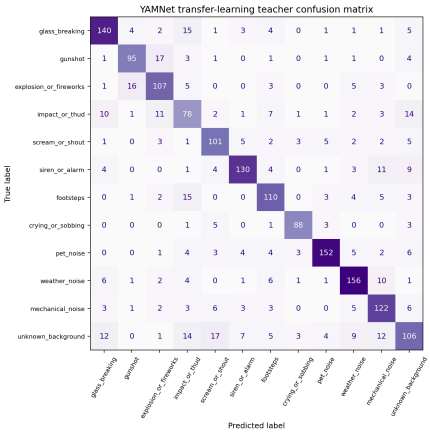
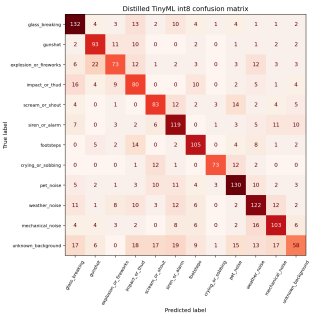
For Colab dataset setup:

```
ESC-50        -> downloaded automatically by the notebook setup cell
UrbanSound8K  -> downloaded automatically by the notebook setup cell
FSD50K        -> upload/extract audio to Google Drive at My
Drive/tinyml_datasets/FSD50K/
Donate-a-cry  -> cloned automatically from GitHub (git clone --depth 1)
```

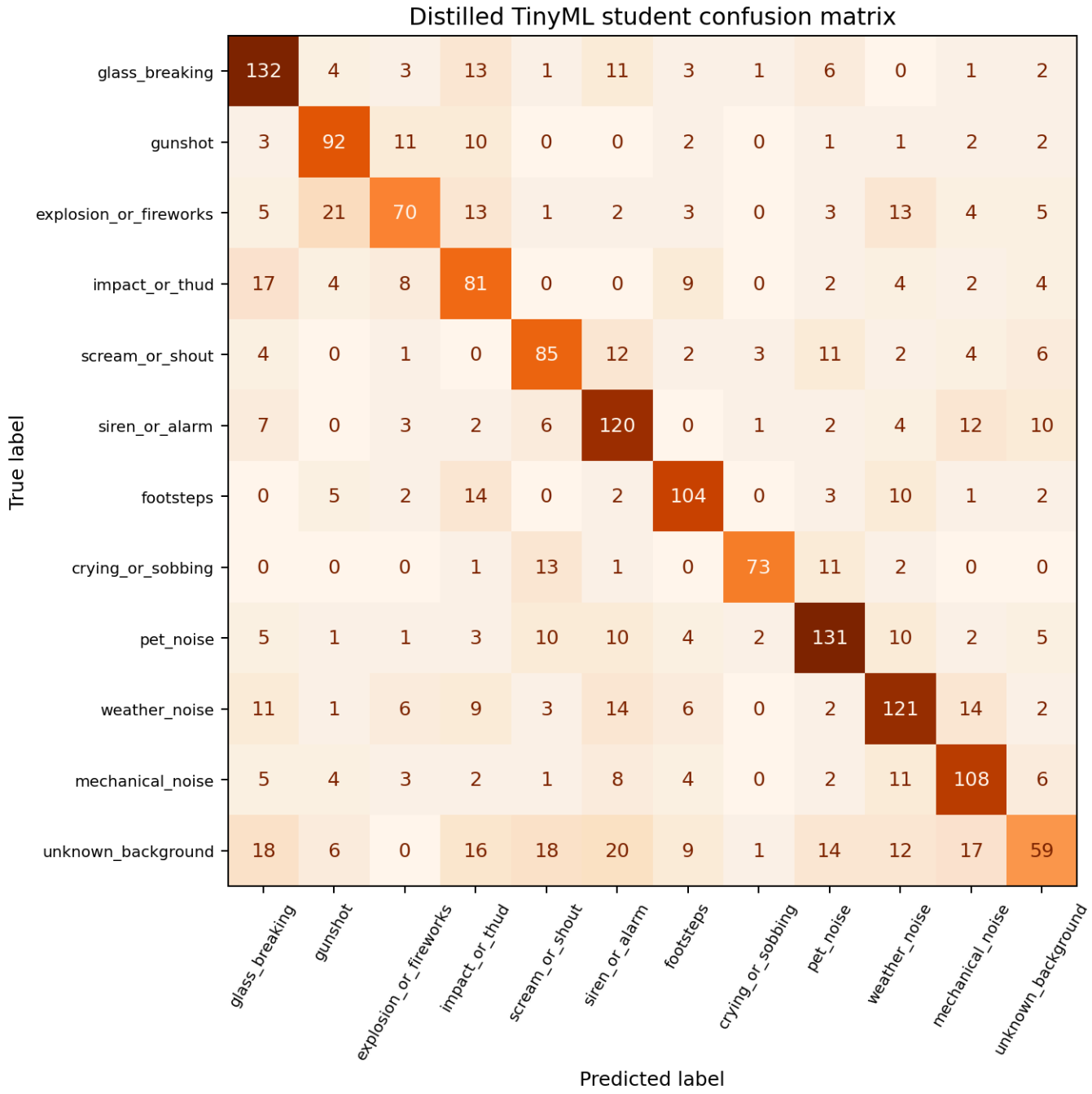
The notebook mounts Google Drive and links that folder to `/content/data/raw/FSD50K` when running in Colab. The FSD50K metadata zip is small and downloaded automatically; only the large audio files need to be provided.

Comparison Table

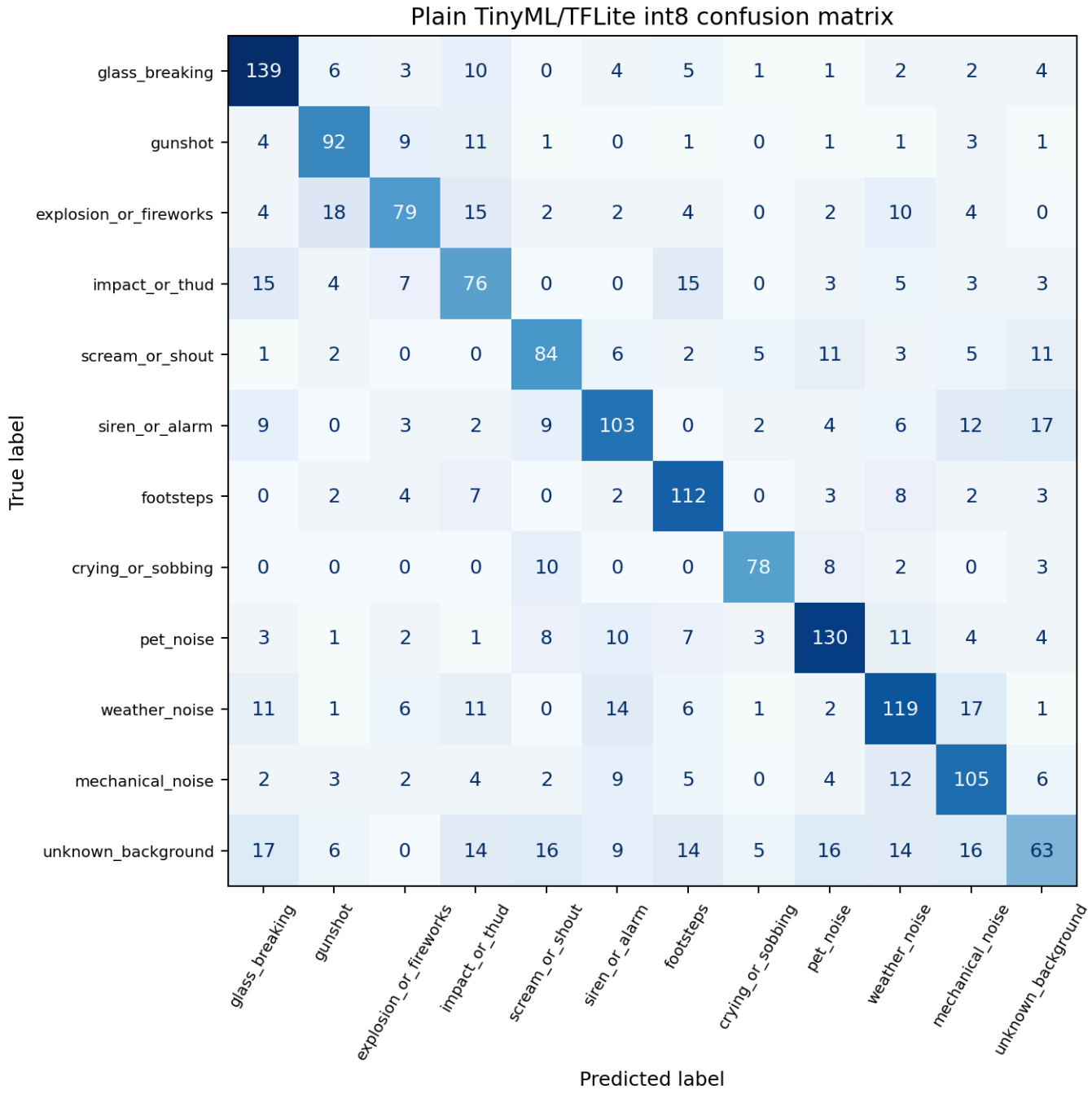
Last completed run: v8. v9 Mixup regression. v10 results pending.

Metric	**Scratch TensorFlow**	**YAMNet Teacher**	**Distilled TinyML Student**
Target device	Laptop / Colab	Laptop / Colab training reference	ESP32 candidate
Numeric format	float32	float32 embeddings/classifier	int8 weights (float32 I/O)
Model size	~2.4 MB (int8: 216 KB)	~13 MB	**64 KB**
Test accuracy (v8)	64.7% (float32) / 64.5% (int8)	75.7%	**66.9%** (int8)
Confusion matrix			
Inference time	~6.6 ms/window (laptop int8)	N/A	~0.4 ms/window (ESP32 est.)
Memory constraint	Low relevance	Too large for ESP32 WROOM	Fits in ESP32 flash
Data sent to openHAB	optional raw/central	not deployed	label + confidence only
Debug data access	notebook arrays/plots	notebook arrays/plots	serial monitor or debug MQTT

Distilled student float32 (before quantization):



Quantized regular model (plain int8 baseline, no distillation):



Chance accuracy with 12 classes is 8.3%. The YAMNet teacher result is not the ESP32 deployment target — it is the reference model whose soft probability output guides student training. The deployable result is the distilled compact student converted to int8 TFLite.

Notable v8 result: the distilled student (66.9% int8, 64 KB) outperforms the scratch regular model (64.7%, 2.4 MB) trained on the same data — 37× smaller and more accurate. This is the core demonstration of knowledge distillation for edge deployment.

Version Training History

Each version is a separate notebook. Key changes and their measured impact on distilled student int8 test accuracy:

Version	Labels	Key Change	Regular	Teacher	Student int8	Student size	Notes
v2	13	Initial student: 32/64/96 SeparableConv, T=5.0, alpha=0.35	47.9%	63.9%	43.5%	41 KB	Baseline
v3	13	SpecAugment on regular model; 70/15/15 random split	60.2%	76.6%	40.0%	41 KB	SpecAugment improved regular model but broke distillation: masked student input vs full-spectrogram teacher soft labels caused loss 16.5 (vs 6.3 in v2)
v4	13	FSD50K full 51K-file extraction; Gaussian noise replaces SpecAugment for student augmentation	57.6%	74.2%	13.4%	41 KB	Class weights from imbalanced regular dataset applied to balanced distillation split — scream weight 14.7×, student collapsed to predicting scream for 430/1455 test examples
v5	13	Uniform weights in distillation; T=3.0 (down from 5.0)	60.2%	74.2%	**57.3%**	41 KB	Fixed weight collapse; best student result to date
v6	12	`household_noise` dissolved; two-stage training (Stage 1: 30 ep hard labels, Stage 2: distillation); label smoothing	60.2%	75.1%	~40%	41 KB	Regression: Stage 1 created hard-label local minimum; Stage 2 LR cut to 7.5e-5 by epoch 12, model stuck at 39–40% for 100 epochs

Version	Labels	Key Change	Regular	Teacher	Student int8	Student size	Notes
v7	12	Reverted to v5 single-stage; corrected FSD50K label names from official website; added ~20 new FSD50K mappings (Music 14 K, Vehicle, etc.)	59.4%	74.5%	58.6%	41 KB	Removed non-existent FSD50K labels (Chainsaw, Baby_cry, Smoke_detector); added high-volume background sources
v8	12	Donate-a-cry corpus (crying: 175→632); cap raised 800→1200; 4th SepConv(128) block	64.7%	75.7%	**66.9%**	64 KB	New best. Student outperforms regular model (66.9% vs 64.7%) on same data — 37× smaller. 8 534 train / 1 830 test examples
v9	12	Mixup augmentation (Beta(0.2,0.2), batch-level, blends spectrograms + teacher soft labels); version-aware Drive checkpoints; 120 ep / patience 18; reuses v8 features + teacher	64.7%	75.7%	64.0%	64 KB	Regression vs v8: student dropped from 66.9% to 64.0%. Mixup alpha=0.2 too aggressive — blended spectrograms may create unrealistic combinations for this dataset. v8 remains best student.
v10	12	Three-phase teacher: Phase 1 frozen embeddings + compact Dense head (256→128, L2=1e-3, Dropout 0.5/0.4); Phase 2 end-to-end fine-tuning via	—	—	—	—	Results pending

Version	Labels	Key Change	Regular	Teacher	Student int8	Student size	Notes
		`hub.load()` @ LR=1e-5 with 3-epoch warmup + gradient clipping (norm=1.0), 25 ep / patience 5; Phase 3 re-extract all embeddings with fine-tuned YAMNet. Regular CNN: `Conv → BatchNorm → ReLU` blocks added. Student: SpecAugment + Gaussian noise augmentation, Dropout(0.35) after Dense(128), distillation T=4.0. Student TFLite: float32 I/O (int8 internal) to avoid Normalization layer scale=0 collapse.					

Key lessons learned:

- SpecAugment broke knowledge distillation in v3: the teacher processed the full spectrogram in the same forward pass as the student, so the teacher's soft labels reflected the un-masked input while the student saw the masked version. The fix (v10) is to pre-compute teacher soft labels on original un-masked spectrograms and store them in the dataset before any augmentation. SpecAugment then augments the student's input on-the-fly during training; the teacher labels remain anchored to the original examples. This is standard distillation practice and avoids the v3 failure.
- Class weights must match the distribution of the dataset they are applied to. The distillation split is intentionally balanced; importing weights computed on the imbalanced regular split amplified minority classes by up to 14.7× and caused class collapse.
- Two-stage training (hard labels then distillation) can trap the student in a local minimum that low-LR distillation cannot escape.
- FSD50K compound label display names (`Crying, sobbing`) map to `_and_` in the ground-truth CSV (`Crying_and_sobbing`), not to comma-separated tokens. Labels not in the official 200-class list (Chainsaw, Baby_cry, Smoke_detector) simply do not exist in FSD50K and should be removed from the mapping.

- Mixup augmentation (v9, alpha=0.2) caused a regression from 66.9% to 64.0%. Blending acoustically diverse spectrograms may create unrealistic combinations that hurt rather than regularise. SpecAugment broke distillation for a different reason (masked input vs full-spectrogram teacher); Mixup broke it by creating blended inputs that neither label describes well.

Expected Interpretation

Commonalities:

- same classification task
- same input concept (log-mel spectrogram)
- same output labels
- same model-training pipeline before conversion

Differences:

- TinyML student model must fit into microcontroller memory
- quantization reduces size and may reduce accuracy
- inference runs locally on the ESP32
- MQTT publishes only summarised results
- local inference improves privacy and reduces bandwidth
- raw microphone samples can still be inspected during development through serial output or temporary debug topics
- teacher-assisted training can improve the small model without deploying the teacher

Important interpretation:

```
The regular/YAMNet models do not need to fit on the microcontroller.
They are training references used to improve and evaluate the compact student.
The deployable edge version is the distilled int8 TFLite/TFLite Micro model.
```

Artifacts To Produce

```
tinyml/models/regular_model.keras
tinyml/models/regular_model_float32.tflite
tinyml/models/tiny_model_int8.tflite
tinyml/models/distilled_student_model.keras
tinyml/models/distilled_student_float32.tflite
tinyml/models/distilled_student_int8.tflite
tinyml/exported/distilled_student_int8.cc
tinyml/exported/figures/*.png
```

The `.keras`, `.tflite`, and exported `.cc` files are generated by the notebook.

Debug Evidence

For the project report, capture evidence that the ESP32 audio node can be inspected without making raw audio part of the normal openHAB data path.

Possible evidence:

- notebook plot of one audio window or spectrogram
- serial monitor screenshot with a short raw sample preview
- MQTT Explorer screenshot of `tele/safety_audio_1/CLASSIFICATION`
- optional MQTT Explorer screenshot of `tele/safety_audio_1/FEATURES`

The explanation should be:

```
Raw or feature-level data is available during development for calibration and
verification.
The deployed MQTT/openHAB integration uses summarised inference data only.
```